

Mathematics for AI

*Note: HW1- Singular Value Decomposition, SVD

이도현
인공지능학과
20231851

Abstract—임의의 해상도 image에 대해서 full SVD, thin SVD, Compact SVD, Truncated SVD를 구현했다. 그 결과를 바탕으로 입력영상의 특성에 따른 singular value를 대각원소로 가지는 matrix의 특성을 Truncated SVD와의 비교로 분석했다. ***SVD 코드가 아닌 RGB 채널별 계산, RMSE/PNSR 계산, U V 에 관한 추가 논의는 GPT를 활용했습니다.***

I. INTRODUCTION

이번 과제의 목표는 다양한 방법의 Singular Value Decomposition (SVD)을 이미지 데이터에 적용하여, singular value가 가지는 특성과 영상 압축에 미치는 영향을 분석하는 것이다. 우선 full SVD, thin SVD, compact SVD, truncated SVD를 각각 설명하고, 이를 실제 영상에 적용한 과정을 코드를 통해 설명한다. 또한, 영상 압축 결과를 비교하고 singular value 행렬의 분포를 분석하며, truncated SVD의 성능을 집중적으로 평가한다.

SVD는 임의의 행렬을 직교 행렬 U , 대각 행렬 Σ , 직교 행렬 V^T 로 분해하는 기법으로, 영상과 같은 2차원 데이터에서 불필요한 정보를 줄이면서도 중요한 특징을 유지할 수 있다. 특히, Σ 의 singular value는 데이터의 에너지 크기를 나타내고, 값이 큰 일부 성분만으로도 원본 영상의 중요한 구조를 복원할 수 있다. 따라서 영상 압축에서 SVD는 정보 손실을 최소화하면서 데이터 크기를 줄이는 방법으로 사용할 수 있다.

추가적으로, 과제와 관련은 없지만 U 와 V 행렬의 열벡터가 가지는 의미를 살펴보고, 이들이 영상의 basis로서 어떤 역할인지 논의할 것이다.

II. METHODS

본 과제에서는 하나의 이미지를 행렬로 표현한 뒤, 네 가지 방식의 SVD를 적용하였다. 각 방법은 동일한 SVD 분해를 기반으로 하지만, 보존하는 차원과 계산 효율성에서 차이를 가진다. 다음에서는 각 방법의 정의와 구현 방식을 설명한다.

A. Full SVD

Full SVD는 임의의 $m \times n$ 행렬 A 를 다음과 같이 세 행렬로 분해한다.

$$A = U \Sigma V^T$$

여기서 U 는 $m \times m$ 직교 행렬, Σ 는 $m \times n$ 대각 행렬, V 는 $n \times n$ 직교 행렬이다. Full SVD는 모든 특이값(singular value)과 대응하는 벡터를 포함하므로, 정보 손실이 전혀 없는 완전한 분해가 가능하다. 하지만 계산량이 크고 불필요한 성분까지 포함할 수 있다.

Identify applicable funding agency here. If none, delete this.

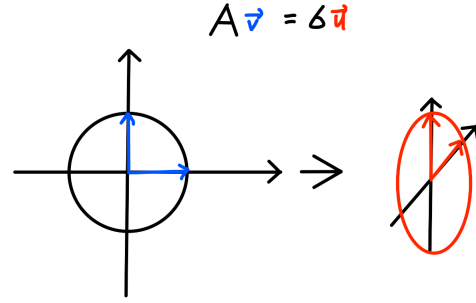


Fig. 1. 행렬 A 를 U, Σ, V^T 로 분해하는 SVD의 직관적 구조. V 의 열벡터가 singular value Σ 로 스케일링되어, U 의 열벡터 방향으로 사상되는 과정을 손그림으로 나타냈다.

B. Thin SVD

Thin SVD는 Full SVD의 불필요한 부분을 제거하여 계산 효율성을 높인다. A 가 $m \times n$ 행렬이고 $m \geq n$ 일 때, U 를 $m \times n$ 으로 줄이고, Σ 를 $n \times n$ 으로 만든다. 즉, 실제 유효한 성분만 남겨서 분해를 수행한다. Thin SVD는 Full SVD와 동일한 정보를 담고 있으나 메모리 사용량과 계산량이 더 적다.

C. Compact SVD

Compact SVD는 rank- r 행렬 A 에 대해 유효한 랭크 r 까지만 성분을 남겨 분해하는 방식이다. 즉, Σ 에서 0이 아닌 singular value만 남기고, 이에 대응하는 U 와 V 의 열벡터만 선택한다. 이 방법은 차원 축소와 데이터 압축에서 효율적이다.

D. Truncated SVD

Truncated SVD는 Compact SVD를 일반화한 형태로, 상위 k 개의 singular value만 보존하고 나머지는 버린다. 즉,

$$A \approx U_k \Sigma_k V_k^T$$

와 같이 근사 행렬을 만든다. 여기서 $k < r$ 일 경우 일부 정보가 손실되지만, 데이터 크기를 크게 줄이면서도 중요한 특징을 유지할 수 있기 때문에 영상 압축에서 활용 가능하다.

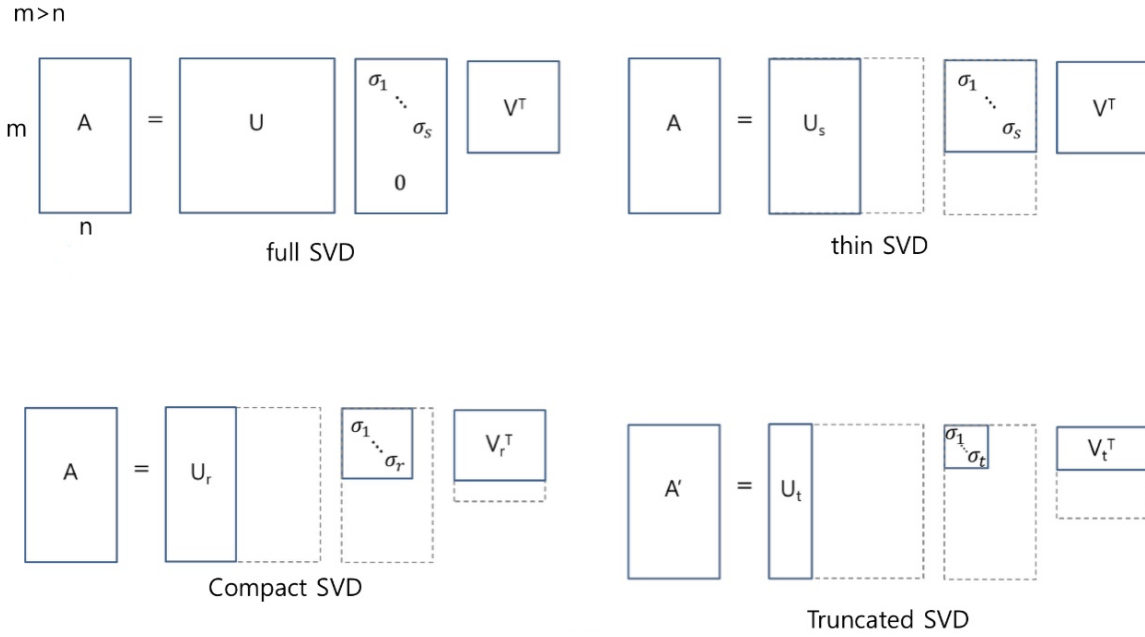


Fig. 2. Full SVD, Thin SVD, Compact SVD, Truncated SVD의 비교. 각 방법은 보존되는 singular value의 개수와 U , V 행렬의 크기에서 차이를 보인다.

III. IMPLEMENTATION DETAILS

과제의 구현은 NumPy와 Pillow, Matplotlib 라이브러리를 사용했다. RGB 채널별로 독립적으로 실행하고, 각 채널은 $[0, 1]$ 범위로 정규화된 실수 행렬로 처리된다. 주요 함수의 구성은 다음과 같다.

- **load_rgb(path)**: 입력 이미지를 RGB 채널로 불러와 실수형 배열로 변환한다.
- **to_uint8(A)**: 계산 결과를 다시 0-255 범위의 8비트 정수 영상으로 변환한다.
- **rmse(A, B)**, **psnr(A, B)**: 복원 영상의 품질을 정량적으로 평가하기 위한 지표로, RMSE와 PSNR을 계산한다.
- **_orthonormalize_columns(X)**: Gram-Schmidt 과정을 통해 입력 행렬의 열벡터를 정규직교화한다.
- **_complete_orthonormal_basis(Q, m)**: 주어진 직교열벡터 집합 Q 를 확장하여 나머지 직교 기저를 찾는다.
- **svd_full(A)**: Full SVD를 수행하여 U , Σ , V^T 를 구한다. 모든 특이값과 대응 벡터를 포함한다.
- **thin_svd(A)**: 실제 유효한 차원($\min(m, n)$)까지만 고려한 SVD를 수행한다.
- **compact_svd(A)**: 0이 아닌 singular value만 보존하여 차원을 축소한 Compact SVD를 실행한다.
- **svd_truncate(A, k)**: 상위 k 개의 singular value만 남기는 Truncated SVD를 수행한다.
- **rgb_truncated_reconstruct(A, k)**: RGB 영상에 대해 각 채널별로 Truncated SVD를 적용하고 복원된 이미지를 합성한다.

IV. FULL / THIN / COMPACT SVD

Full, Thin, Compact SVD는 그저 수학적 분해 방식이다. (부동 소수점 계산에 의한 RMSE 범위보다 작은 rank에 대한 오차는 무시한다.) 즉, 원본 행렬 $A \in \mathbb{R}^{m \times n}$ 을 다음과 같이 분해했을 때,

$$A = U\Sigma V^T,$$

여기서

$$U \in \mathbb{R}^{m \times m}, \quad V \in \mathbb{R}^{n \times n}, \quad \Sigma \in \mathbb{R}^{m \times n},$$

U 와 V 는 각각 직교행렬(orthogonal matrix)이며, Σ 는 특이값(singular value)을 대각 원소로 갖는 대각행렬이다.

따라서 이 세 행렬을 다시 곱하면,

$$U\Sigma V^T = A,$$

가 항상 성립한다.

위와 같은 식으로 복원을 진행하지만 만약 특이값이 0인 행렬이 있다면 그 행렬에 대한 내용을 지워버리기 때문에 U 와 V 직교행렬에 대한 정보가 일부 사라졌을 것이다.

하지만 실험 결과를 보면,

Full SVD $\rightarrow$ RMSE = 0.000000, PSNR = 212.16 dB,

Thin SVD $\rightarrow$ RMSE = 0.000000, PSNR = 212.16 dB,

Compact SVD $\rightarrow$ RMSE = 0.000000, PSNR = 212.16 dB.

인 것을 볼 수 있다. (Fig 3)에서 볼 수 있듯이 매우 작은 특이값이지만 값을 가지고 있는 것을 볼 수 있다.

$$\Sigma = \begin{bmatrix} 601.9743 & 0 & 0 & \cdots & 0 \\ 0 & 78.0970 & 0 & \cdots & 0 \\ 0 & 0 & 70.6747 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 3.9748 \end{bmatrix}$$

즉, 모든 column이 선형 독립이기에 모든 특이값이 0이 아니고, 손실된 column이 없어서 복원 가능하게 나온 것이다. 만약 하나의 픽셀 값도 달라지지 않는 선형 의존 col이 존재한다면 오차가 생겼을 것이다. 즉, 이 이미지는 여러 종류의 SVD특징을 보기엔 어렵지만 선형 독립과 SVD의 관계를 볼 수 있다.

V. TRUNCATED SVD

A. Singular Value Matrix

RGB 각 채널의 특이값을 산출하고, 동일 순서의 특이값을 평균하여 영상 전체의 대표 스펙트럼을 구성하였다. 평균 특이값은 다음과 같이 정의된다:

$$\bar{\sigma}_i = \frac{\sigma_i^{(R)} + \sigma_i^{(G)} + \sigma_i^{(B)}}{3}, \quad i = 1, 2, \dots$$

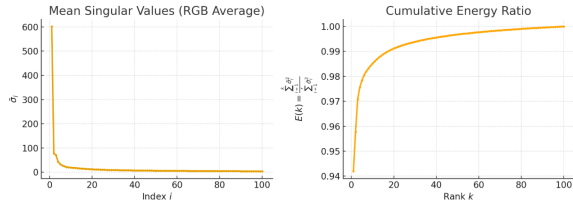


Fig. 3. RGB 평균 특이값 스펙트럼(왼쪽)과 누적 에너지 비율(오른쪽). 초기 특이값 $\bar{\sigma}_1$ 이 이후 값들보다 압도적으로 크며, 상위 소수의 특이값만으로 전체 에너지의 대부분이 유지됨을 확인할 수 있다. 이는 영상의 정보가 저차원 공간에 집중되어 있음을 의미한다.

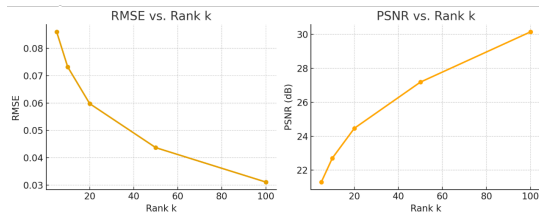


Fig. 4. Truncated SVD의 Rank k 에 따른 RMSE 및 PSNR 변화. 초기 랭크 구간($k = 5, 10$) RMSE 감소율과 PSNR 증가율이 모두 더 크며, 이후 구간에서는 기울기가 점차 완만해지는 것을 확인할 수 있다.

TABLE I
TRUNCATED SVD의 RANK k 에 따른 RMSE 및 PSNR 변화

Rank k	RMSE	PSNR (dB)
5	0.086051	21.30
10	0.073233	22.71
20	0.059779	24.47
50	0.043678	27.19
100	0.031082	30.15



Fig. 5. Truncated SVD에 따른 복원 결과 비교

B. Result Analysis

Truncated SVD는 상위 k 개의 특이값만을 보존하므로, k 가 작을수록 세부 정보가 손실되지만 영상의 주요 구조나 형태적 특징은 여전히 유지된다. Table I에서 볼 수 있듯이, Rank k 를 5에서 100으로 증가시킬 때 RMSE는 0.086에서 0.031로 감소하고 PSNR은 21.3 dB에서 30.15 dB로 상승하였다. 이는 고차 특이값이 점차 세부적인 명암이나 질감 정보를 담당함을 보여준다.

또한 평균 특이값 스펙트럼(Fig 3)은 초기 몇 개의 특이값이 매우 큰 반면 이후 항들이 급격히 작아지는 형태를 보였다. 이는 영상의 주요 정보가 앞쪽 주요 singular component에 집중되어 있음을 의미하며, Truncated SVD가 낮은 차원에서 어느정도 높은 품질의 복원이 가능한 이유를 설명한다. (Fig 5)를 보면 그 차이를 극명하게 알 수 있다.

VI. CONCLUSION

본 과제에서는 Full, Thin, Compact, Truncated SVD를 구현하고 RGB 영상에 적용하여 그 수학적, 실험적 특성을 분석하였다. 그림 4에서도 확인되듯이, 초기 구간($k = 5, 10$)에서는 RMSE의 급격한 감소와 PSNR의 급상승이 발생하나, $k \geq 50$ 이후부터는 변화가 점차 완만해진다. 즉, Singular value를 적절히 선택함에 따라 SVD가 영상 압축에서 효과적으로 쓰일 수 있다는 것을 알았다.

VII. 부록

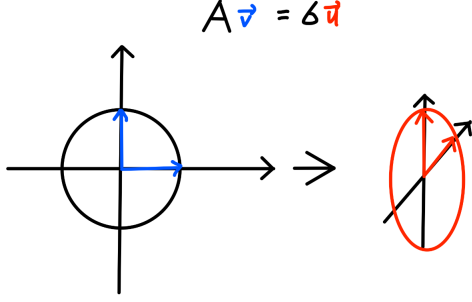


Fig. 6. 행렬 A 를 U, Σ, V^T 로 분해하는 SVD의 직관적 구조. V 의 열벡터가 singular value Σ 로 스케일링되어, U 의 열벡터 방향으로 사상되는 과정을 손그림으로 나타냈다.

A. U, V 벡터의 코사인 유사도에 따른 Singular Value 분석

본 실험에서는 SVD 분해로부터 얻은 U, V 벡터 쌍 각각의 코사인 유사도를 계산하고, 그 유사도가 큰 순서 또는 작은 순서대로 정렬하여 상위 k 개의 성분을 선택하였다. 이는 일반적인 singular value 크기 기반의 truncation과 달리, 벡터 방향의 일치성(혹은 다양성)에 초점을 맞춘 선택 방법이다.

$$\cos \theta_i = \frac{u_i^T v_i}{\|u_i\|_2 \|v_i\|_2} \quad (1)$$

코사인 유사도가 큰 성분은 U 와 V 의 방향이 유사하여 주로 영상의 전체적인 밝기나 평탄한 구조 정보를 보존하는 반면, 유사도가 작은 성분은 방향이 상이하여 텍스처나 세부적인 경계 정보를 포함하는 경향이 있다. 이에 따라 두 가지 경우(작은 각도 vs 큰 각도)를 각각 k 단계별로 비교하였다.

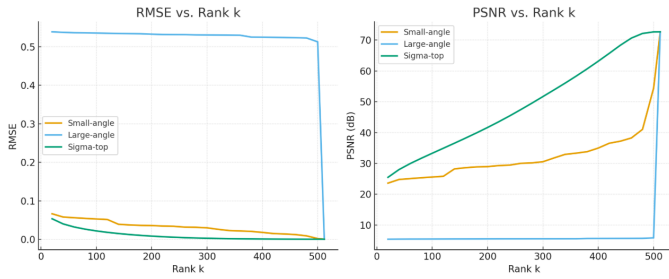


Fig. 7. U - V 벡터의 코사인 유사도에 따라 top- k singular value를 선택한 결과. Small-angle 선택은 전체 구조를 우선적으로 복원하며, Large-angle 선택은 세부 텍스처 복원이 미약함을 보여준다.

그 결과, 코사인 유사도 기반 선택은 σ 크기 기반 Truncated SVD 보다 RMSE 관점에서 상대적으로 비효율적이었으나, 영상의 방향성 구조나 특정 패턴 강조에 활용할 수 있는 잠재력을 보였다. 이는 directional diversity maximization 접근으로 해석될 수 있으며, CNN이나 Transformer에서의 outlier 노이즈 제거에 활용될 수 있을 것이다.

<https://github.com/theholes/svd.git> 에 전체 사진자료와 코드를 올려두었다.